CS 3244
Machine Learning

12

Week 12 Post-Class
Markov Decision Processes

Kan Min-Yen

Slides adapted from David Silver's (UCL) RL course

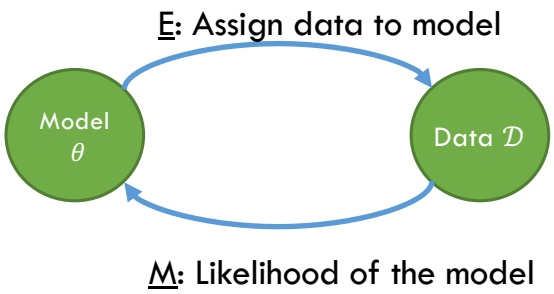
Background Photo credits: Rafiq Mirza, Luan Ahn, Rosmarie Voegtli @ Flickr

Recap and Forecast

Dimensionality Reduction: Find a low rank approximation of the original data. Use orthonormal bases with the most variance to determine principal components.

Use SVD to achieve PCA for practicality.

Clustering: (Iterative) learning without labels by assuming guesses are correct answers.
Hard: K means, Soft: GMM.



Learning Outcomes for this week:

- Understand the basic anatomy and terminology in reinforcement learning
  - Policy, reward, action, observation and (environment, agent) model
- Introduce the Markov property and use this to define a Markov decision process (MDP)
- Understand the Bellman equation as a recursive form for calculating future rewards.

Markov Processes

CS3244 Machine Learning

Introduction to Markov Decision Processes

MDPs processes formally describe an environment for reinforcement learning:

where the environment is fully observable: the current state completely characterises the process

- Almost all RL problems can be formalised as MDPs, e.g.
  - Optimal control primarily deals with continuous MDPs
  - Partially observable problems can be converted into MDPs
  - Bandits are MDPs with one state

Markov Property

An information state (a.k.a. Markov state) contains all useful information from the history.

Definition: A state s_t s Markov if and only if

$$P(s_{t-1}|s_t) = P(s_{t+1}|s_1, \dots, s_t)$$

"The future is independent of the past given the present"

$$h_{1:t} \rightarrow \boldsymbol{s_t} \rightarrow h_{t+1:\infty}$$

Once the state is known, the history is irrelevant; *i.e.*,

- The state is a sufficient statistic of the future
- The environment state s_t^e is Markov
- The history h_t is Markov

А. А. Марков (1886).

Andrey Markov

Photo from Wikipedia Commons: Andrey Markov

State Transition Matrix

For a Markov state s and successor state s', the *state transition probability* is defined by

$$\mathcal{P}_{s,s'} = P(s_{t+1} = s' | s_t = s)$$

State transition matrix P defines transition probabilities from all states s to all successor states s',

$$\mathcal{P} = \begin{bmatrix} \mathcal{P}_{1,1} & \cdots & \mathcal{P}_{1,n} \\ \vdots & \ddots & \vdots \\ \mathcal{P}_{n,1} & \cdots & \mathcal{P}_{n,n} \end{bmatrix}$$

where each row of the matrix sums to 1.

Markov Chains

A Markov process is a memoryless random process,
i.e., a sequence of random states $s_1, s_2, \ldots$ with the Markov property.

Definition:

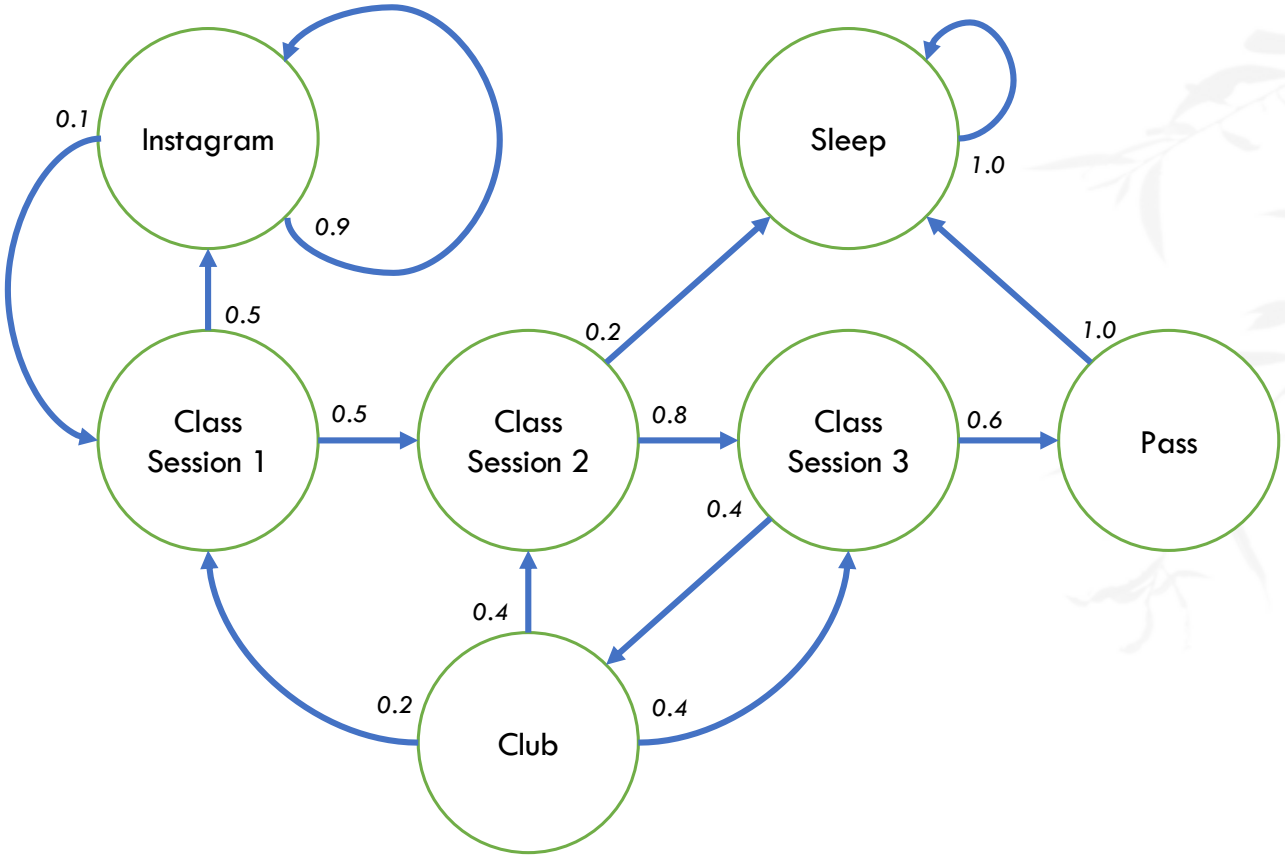
A *Markov Process* (or *Markov Chain*) is a tuple $\langle \mathcal{S}, \mathcal{P} \rangle$

- $\mathcal{S}$ is a (finite) set of states
- $\mathcal{P}$ is a state transition probability matrix, $\mathcal{P}_{s,s'} = P[\mathcal{S}_{t+1} = s' | \mathcal{S}_t = s]$

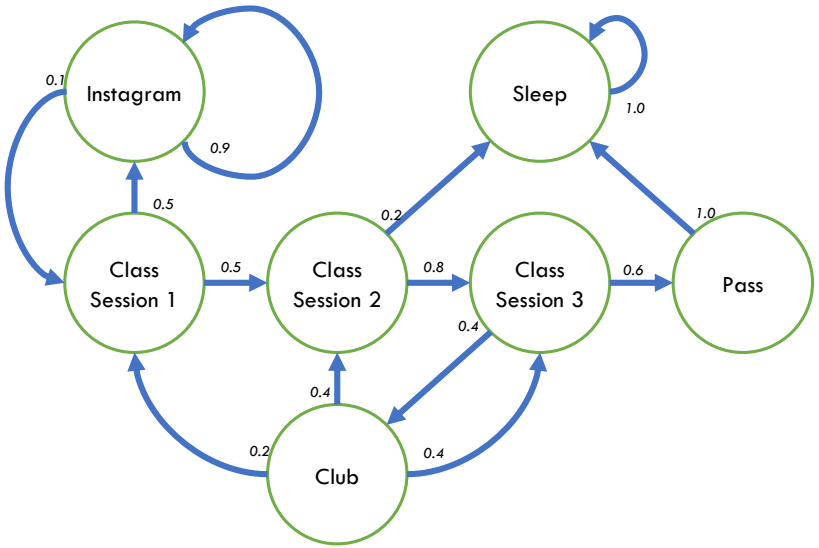
Example Markov Chain

CS3244 Machine Learning

Student Markov Chain Example



Student Markov Chain Example

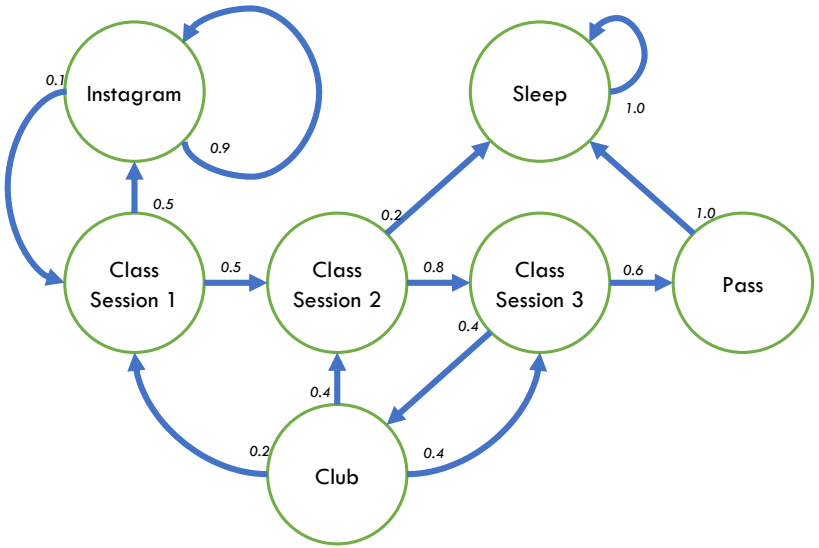


Sample episodes starting from s_1 = Class Session 1 (C1)

$$s_1, s_2, \dots s_T$$

- C1 C2 C3 Pass Sleep
- C1 IG IG C1 C2 Sleep
- C1 C2 C3 Pub C2 C3 Pass Sleep
- C1 IG IG C1 C2 C3 Pub C1 IG IG C1 C2 C3 Pub … Pass Sleep

Student Markov Chain Example



To

$$\mathcal{P} = \begin{array}{c} \\ C1 \\ C2 \\ C3 \\ Pass \\ Club \\ IG \\ Sleep \end{array} \begin{array}{c} \begin{array}{ccccccc} C1 & C2 & C3 & Pass & Club & IG & Sleep \end{array} \\ \begin{bmatrix} & 0.5 & & & & 0.5 & \\ & & 0.8 & & & & 0.2 \\ & & & 0.6 & 0.4 & & \\ & & & & & & 1.0 \\ 0.2 & 0.4 & 0.4 & & & & \\ 0.1 & & & & & 0.9 & \\ & & & & & & 1.0 \end{bmatrix} \end{array}$$

From

Markov Reward Processes

CS3244 Machine Learning

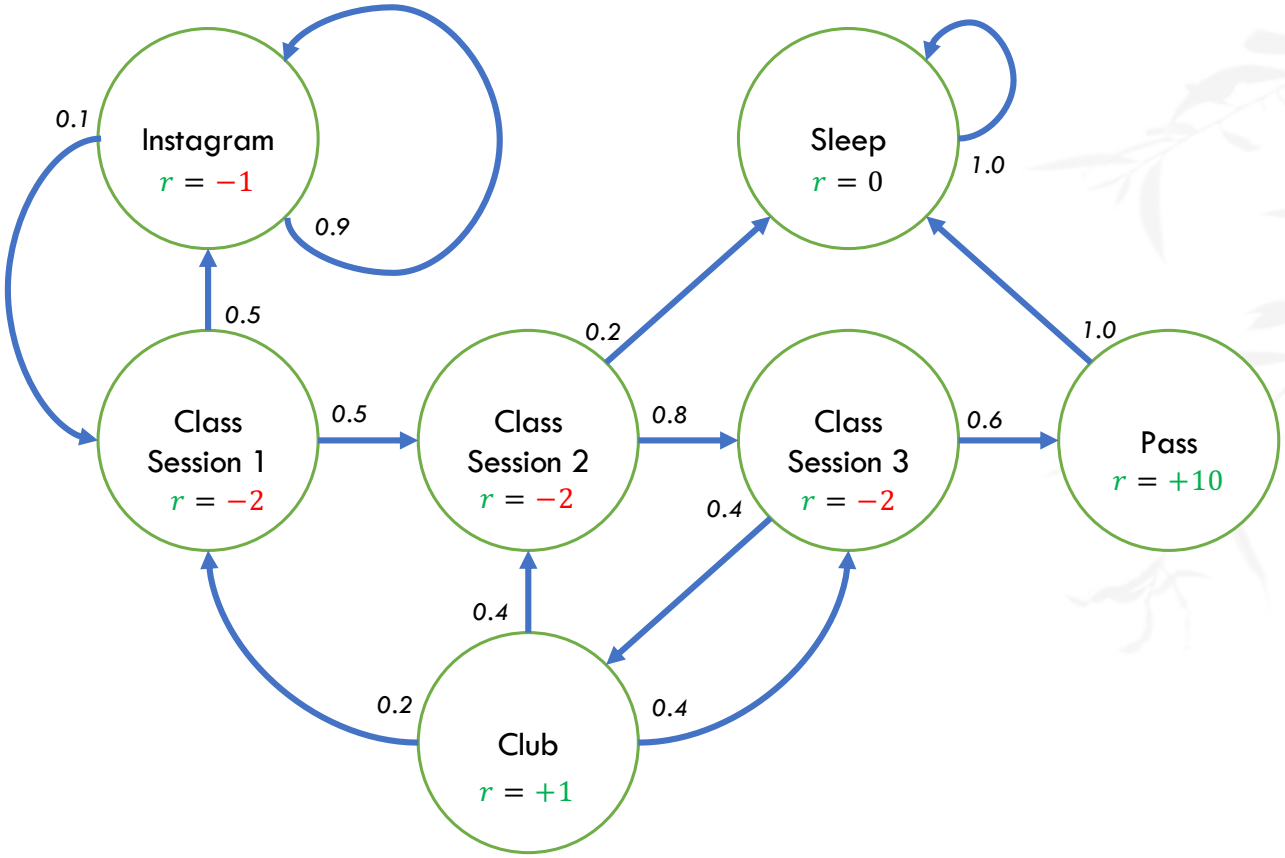
Markov Reward Process

A Markov reward process is a Markov chain with values.

A Markov Reward Process is a tuple $\langle \mathcal{S}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

- $\mathcal{S}$ is a finite set of states
- $\mathcal{P}$ is a state transition probability matrix $\mathcal{P}_{s,s'} = P(s_{t+1=s'} | s_t = s)$
- $\mathcal{R}$ is a reward function, $\mathcal{R}_s = \mathbb{E}(r_{t+1} | s_t = s)$
- γ is a discount factor, $\gamma \in [0,1]$

Student Markov Reward Process



Return

The return g_t is the total discounted reward from time step t.

$$g_t = r_{t+1} + \gamma r_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

- The discount $\gamma \in [0, 1]$ is the present value of future rewards.
- The value of receiving reward r after $k + 1$ time-steps is $\gamma^k r$.
- This values immediate reward above delayed reward.
  - γ close to 0 leads to "near-sighted" evaluation
  - γ close to 1 leads to "far-sighted" evaluation

Why Discount?

Most Markov reward and decision processes are discounted. Why?

- Mathematically convenient to discount rewards
- Avoids infinite returns in cyclic Markov processes
- Uncertainty about the future may not be fully represented
- If the reward is financial, immediate rewards may earn more interest than delayed rewards
- Animal/human behaviour shows preference for immediate reward
- It is sometimes possible to use *undiscounted* Markov reward processes (i.e. $\gamma = 1$), if all sequences terminate.

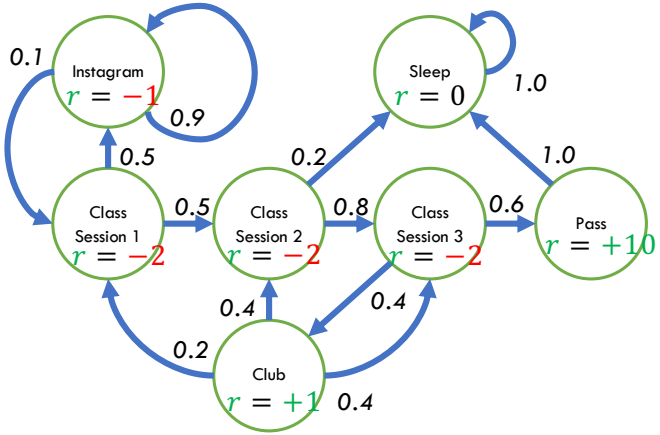
Value Function

The value function $v(s)$ gives the long-term value of state s.

The state value function $v(s)$ of an MRP is the expected return starting from state s:

$$v(s) = \mathbb{E}(g_t | s_t = s)$$

Sample returns from the Student MRP



Starting from $s_1 =$ ____ , with $\gamma =$ ____.

$$g_1 = r_2 + \gamma r_3 + \gamma^2 r_4 + \cdots + \gamma^{T-2} r_T$$

C1 C2 C3 Pass Sleep $\quad g_1 = -2 + \frac{1}{2}\times -2 + \frac{1}{4}\times -2 + \frac{1}{8}\times 10 \quad = -2.25$

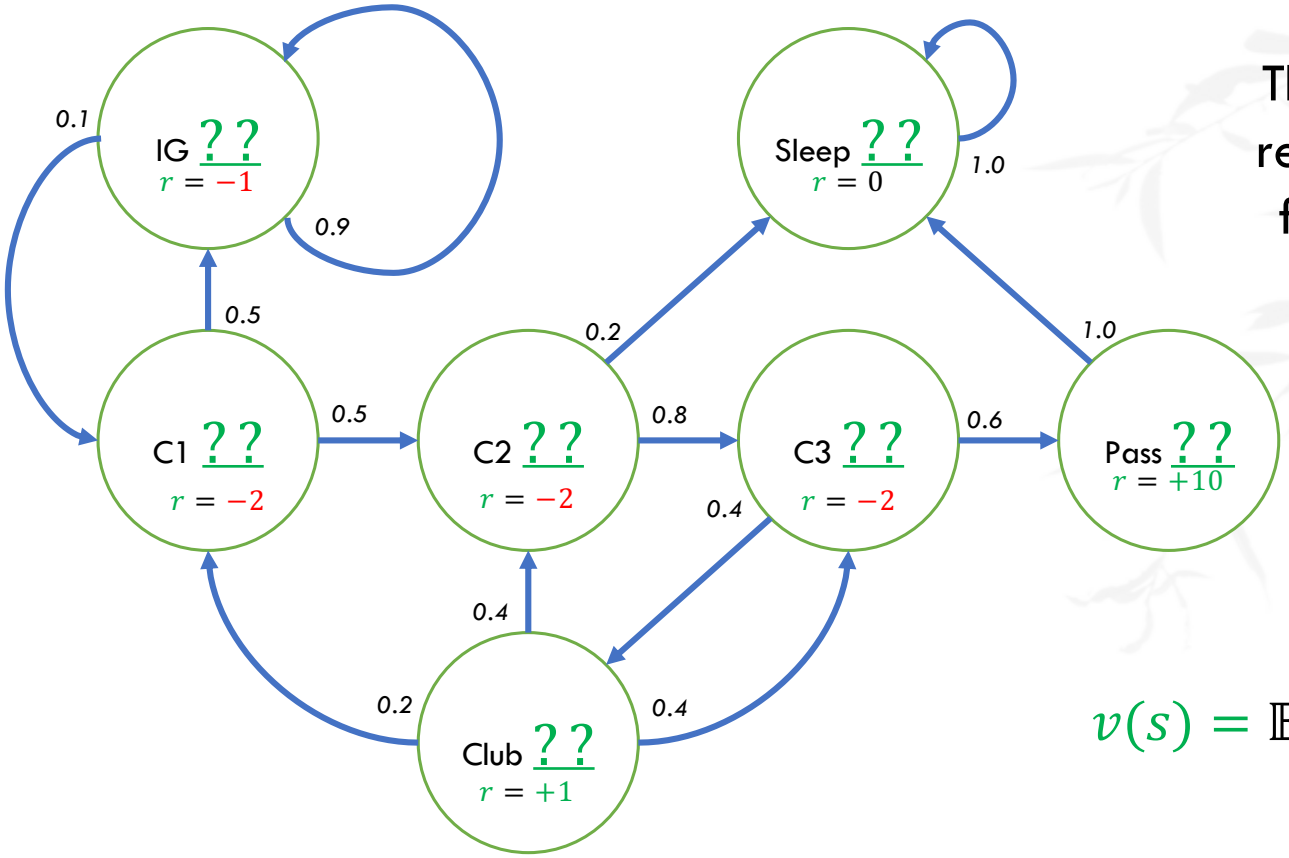
C1 IG IG C1 C2 Sleep $\quad g_1 = -2 + \frac{1}{2}\times -1 + \frac{1}{4}\times -1 + \frac{1}{8}\times -2 + \frac{1}{16}\times -2 \ldots \quad = -3.125$

C1 C2 C3 Pub C2 C3 Pass Sleep $\quad g_1 = -2 + \frac{1}{2}\times -2 + \frac{1}{4}\times -2 + \frac{1}{8}\times 1 + \frac{1}{16}\times -2 \ldots \quad = -3.41$

C1 IG IG C1 C2 C3 Pub C1 … $\quad g_1 = -2 + \frac{1}{2}\times -1 + \frac{1}{4}\times -1 + \frac{1}{8}\times -2 + \frac{1}{16}\times -2 \ldots \quad = -3.20$

IG IG IG C1 C2 C3 Pub C2 Sleep

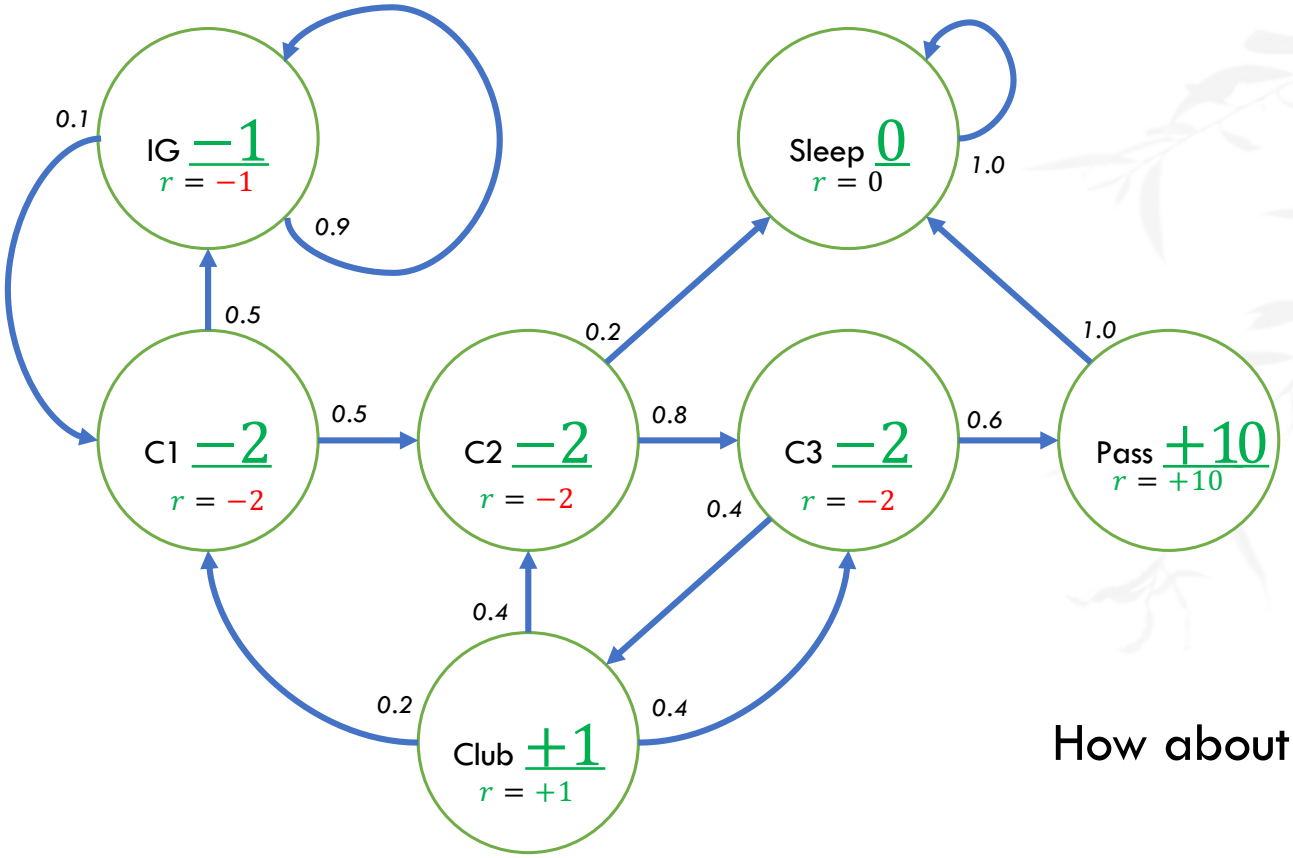
State-Value Function for $\gamma = 0$



The expected return starting from state s

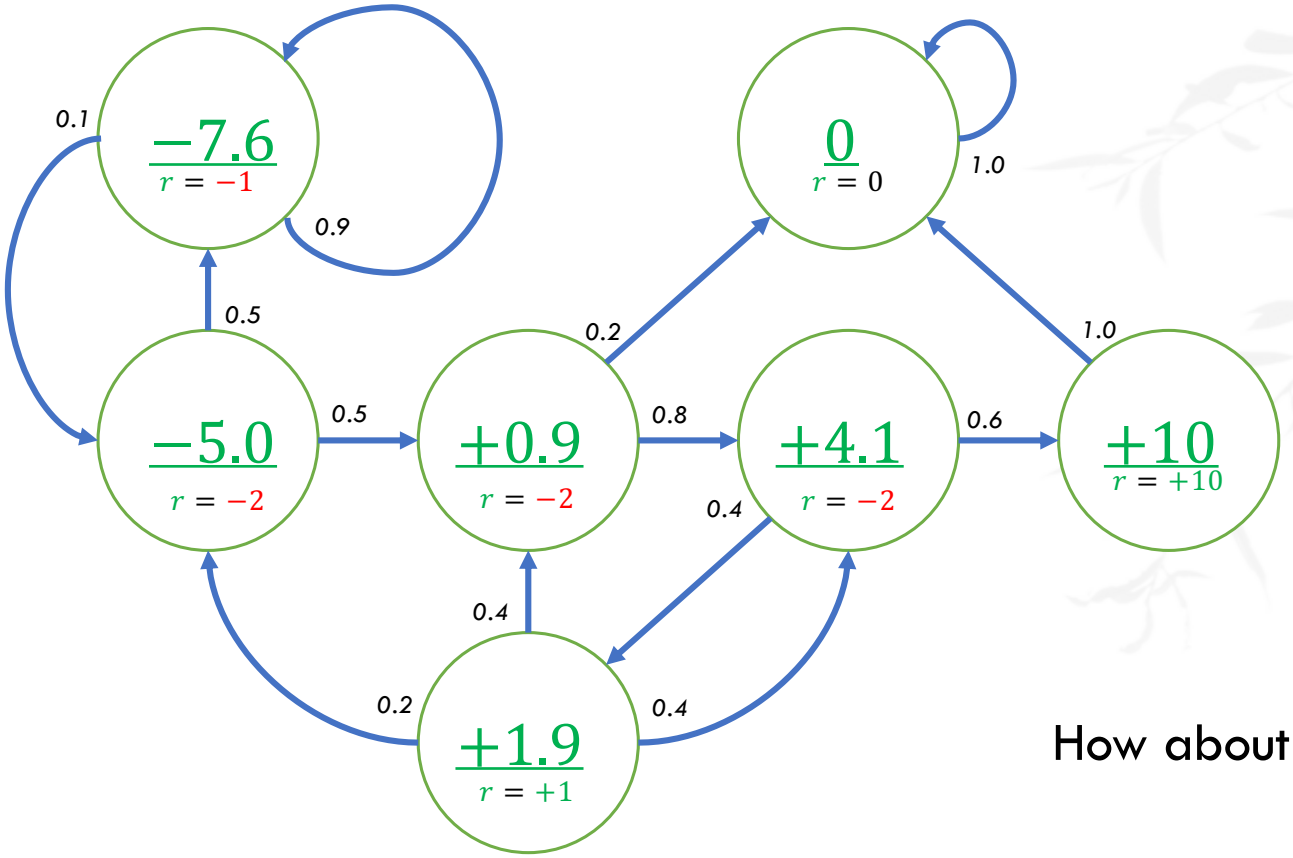
$$v(s) = \mathbb{E}(g_t | s_t = s)$$

State-Value Function for $\gamma = 0$



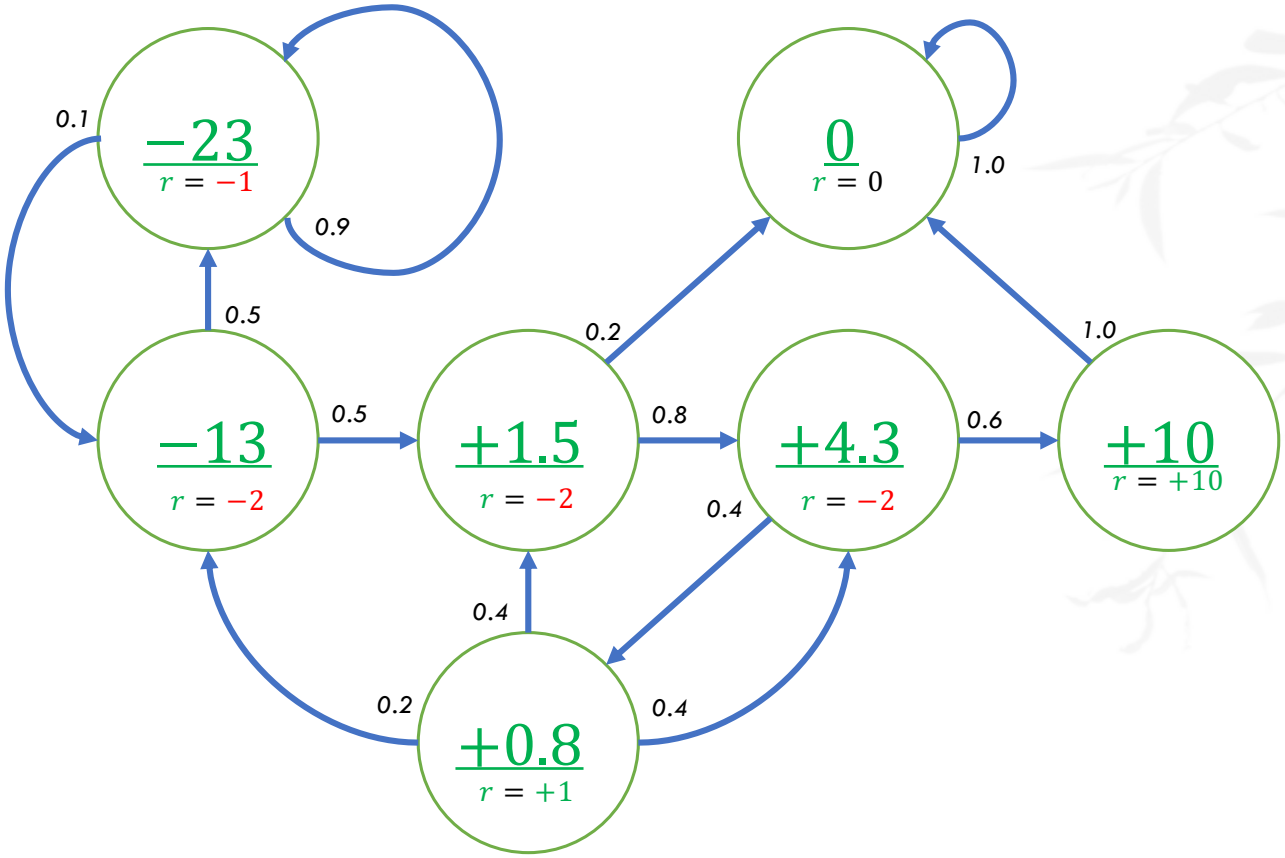
How about for $\gamma = 0.9$?

State-Value Function for $\gamma = 0.9$



How about for $\gamma = 1$?

State-Value Function for $\gamma = 1$



Bellman Equation for MRPs

The value function can be decomposed into two parts:

- immediate reward r_{t+1}
- discounted value of successor state $\gamma v(s_{t+1})$

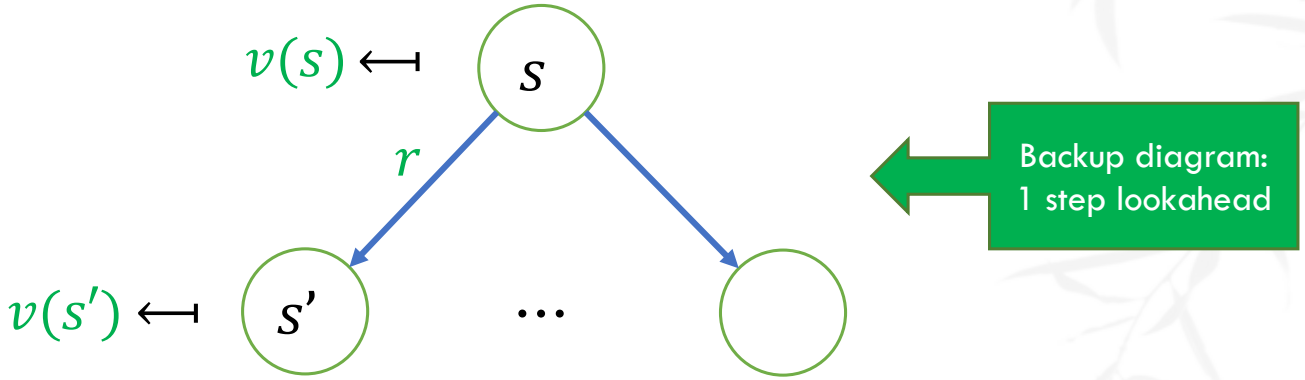
$$
\begin{aligned}
v(s) &= \mathbb{E}(g_t | s_t = s) \\
&= \mathbb{E}(r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots | s_t = s) \\
&= \mathbb{E}(r_{t+1} + \gamma (r_{t+2} + \gamma r_{t+3} + \cdots) | s_t = s) \\
&= \mathbb{E}(r_{t+1} + \gamma g_{t+1} | s_t = s) \\
&= \mathbb{E}(r_{t+1} + \gamma v(s_{t+1}) | s_t = s)
\end{aligned}
$$

By convention, we index time steps to receive the action and let the environment react:

The environment:
- Receives action a_t from the agent
- Emits observation o_{t+1}
- Emits scalar reward r_{t+1}

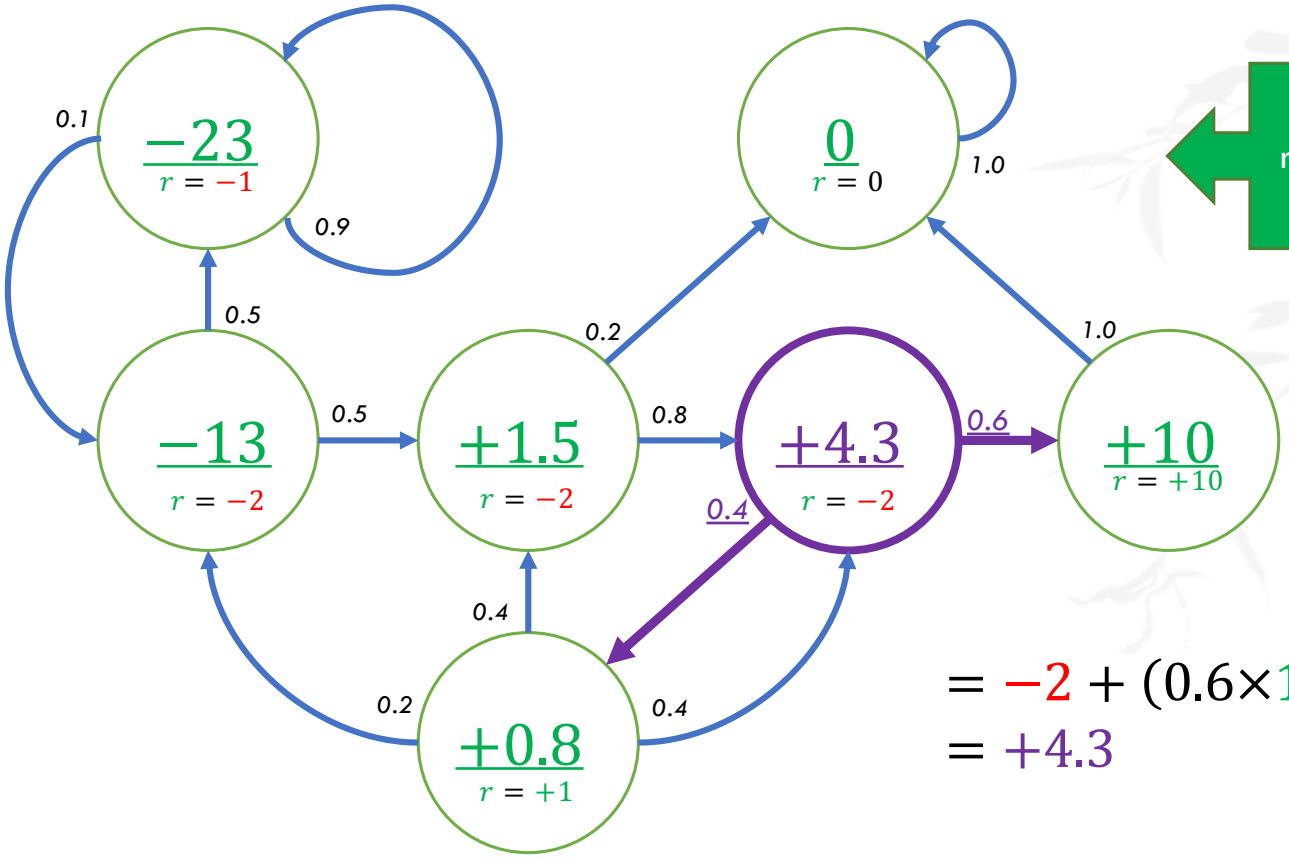
Bellman Equation for MRPs

$$v(s) = \mathbb{E}(r_{t+1} + \gamma v(s_{t+1}) | s_t = s)$$



$$v(s) = \mathcal{R}_s + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{s,s'} \, v(s')$$

Bellman Equation for C3



Remember: receive reward after exiting state regardless of action

$$= -2 + (0.6 \times 10) + (0.4 \times 0.8)$$
$$= +4.3$$

Bellman Equation in Matrix Form

The Bellman equation can be expressed concisely using matrices

$$v = \mathcal{R} + \gamma \mathcal{P} v$$

where v is a column vector with one entry per state:

$$\begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix} = \begin{bmatrix} \mathcal{R}_1 \\ \vdots \\ \mathcal{R}_n \end{bmatrix} + \gamma \begin{bmatrix} \mathcal{P}_{1,1} & \cdots & \mathcal{P}_{1,n} \\ \vdots & \ddots & \vdots \\ \mathcal{P}_{n,1} & \cdots & \mathcal{P}_{n,n} \end{bmatrix} \begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix}$$

Solving the Bellman Equation

The Bellman equation is a linear equation. It can be solved directly:

$$
\begin{aligned}
v &= \mathcal{R} + \gamma \mathcal{P} v \\
(I - \gamma \mathcal{P}) v &= \mathcal{R} \\
v &= (I - \gamma \mathcal{P})^{-1} \mathcal{R}
\end{aligned}
$$

Computational complexity is $O(n^3)$ for n states
Direct solution only possible for small MRPs

There are many iterative methods for large MRPs, e.g.

- Dynamic programming
- Monte-Carlo evaluation
- Temporal-Difference learning

Markov Decision Processes

CS3244 Machine Learning

Markov Decision Process

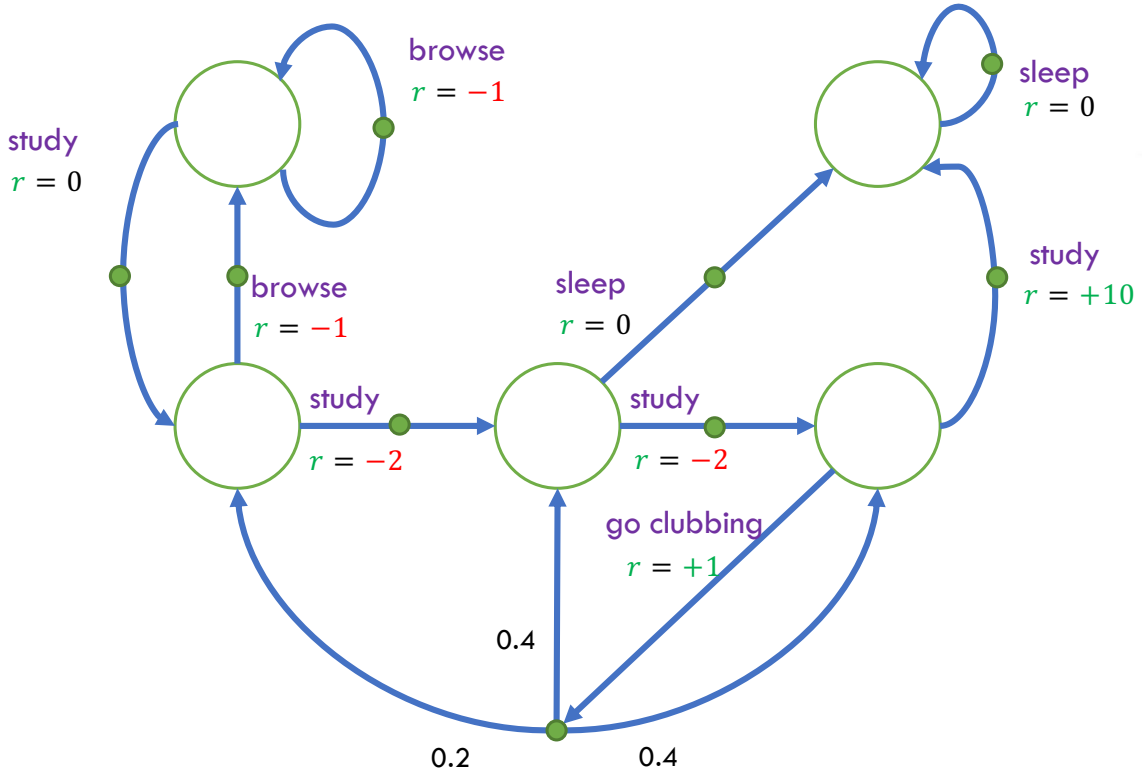
A Markov decision process (MDP) is a Markov reward process with decisions.
It is an *environment* in which all states are Markov.

MDP $\equiv \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

- $\mathcal{S}$ is a finite set of states
- $\mathcal{A}$ is a finite set of actions
- $\mathcal{P}$ is a state transition probability matrix, $\mathcal{P}_{s,s'}^{a} = P(s_{t+1} = s' | s_t = s, a_t = a)$
- $\mathcal{R}$ is a reward function, $\mathcal{R}_s^a = \mathbb{E}(r_{t+1} | s_t = s, a_t = a)$
- γ is a discount factor $\gamma \in [0, 1]$.

In MDP, rewards are indexed on action as well as state. Actions can lead to differing rewards (although not used here).

Student Markov Reward Process



By convention, we leave out action nodes where there is only one action.

⚠️⚠️ This diagram is not equivalent to the previous ones, as the C3 study arc drops the -2 reward.

Policies

A policy π is a distribution over actions given states,

$$\pi(a|s) = P(a_t = a|s_t = s)$$

- A policy fully defines the behaviour of an agent
- MDP policies depend on the current state (not the history)
- i.e. Policies are stationary (time-independent):

$$a_t \sim \pi(\cdot\,|s_t), \forall t > 0$$

Flattening an MDP to an MRP

Given an MDP $m = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ and a policy π

- The state sequence $s_1, s_2, \ldots$ is a Markov process $\langle \mathcal{S}, \mathcal{P}^\pi \rangle$
- The state and reward sequence $s_1, r_2, s_2, \ldots$ is a Markov reward process $\langle \mathcal{S}, \mathcal{P}^\pi, \mathcal{R}^\pi, \gamma \rangle$ where:

$$\mathcal{P}^\pi_{s,s'} = \sum_{a \in \mathcal{A}} \pi(a|s) \mathcal{P}^a_{s,s'}$$

$$\mathcal{R}^\pi_s = \sum_{a \in \mathcal{A}} \pi(a|s) \mathcal{R}^a_s$$

Value Functions of an MDP

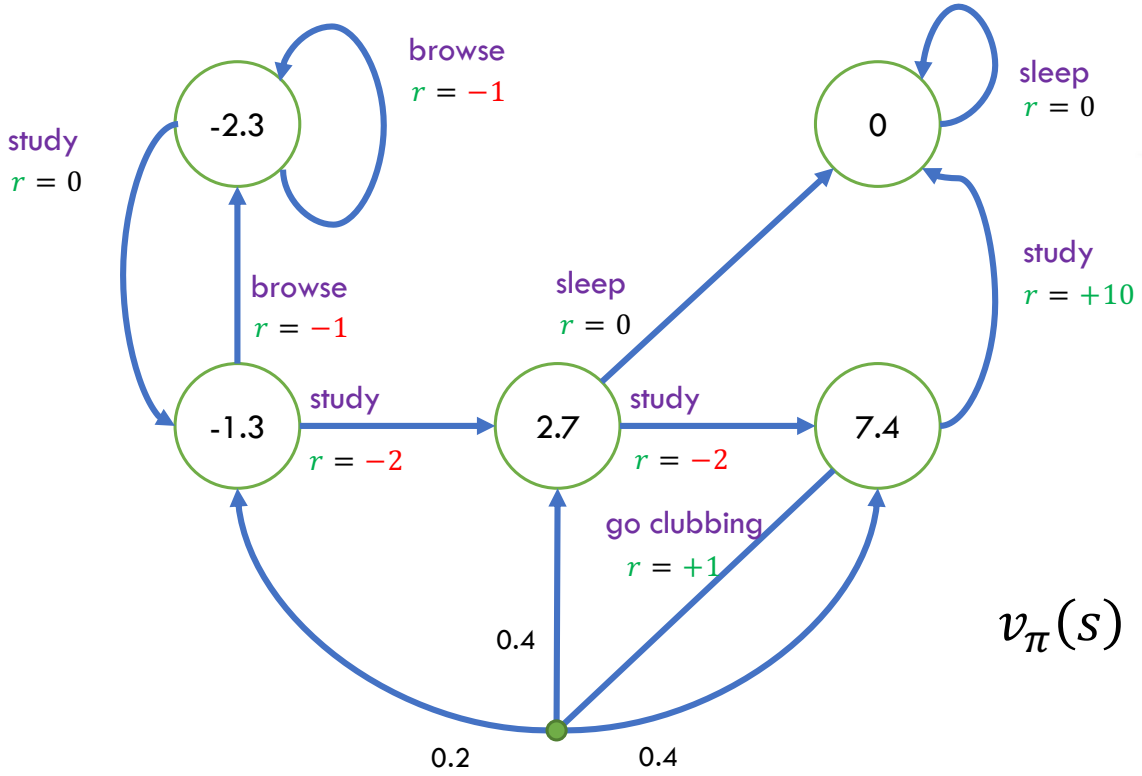
- The *state-value function* $v_\pi(s)$ of an MDP is the expected return starting from state s, and then following policy π:

$$v_\pi(s) = \mathbb{E}_\pi(g_t \mid s_t = s)$$

- The *action-value function* $q_\pi(s, a)$ is the expected return starting from state s, taking action a, and then following policy π:

$$q_\pi(s, a) = \mathbb{E}_\pi(g_t \mid s_t = s, a_t = a)$$

Example Student State-Value Function



$v_\pi(s)$ for $\pi(a|s) = 0.5, \gamma = 1$

Bellman Expectation

CS3244 Machine Learning

Bellman Expectation Equation

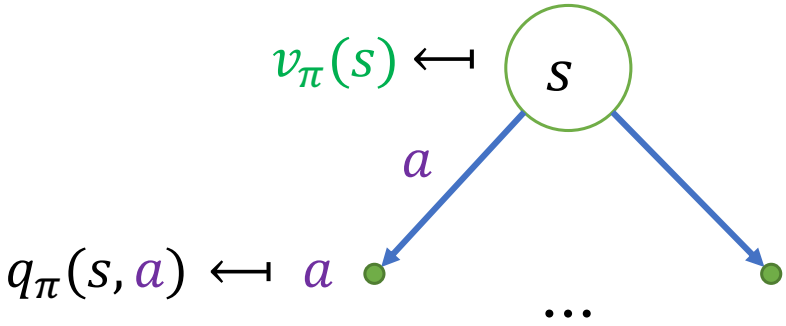
The state-value function can again be decomposed into immediate reward plus discounted value of successor state:

$$v_\pi(s) = \mathbb{E}_\pi\left(\mathcal{R}_{t+1} + \gamma v_\pi(s_{t+1}) | s_t = s\right)$$

The action-value function can similarly be decomposed:

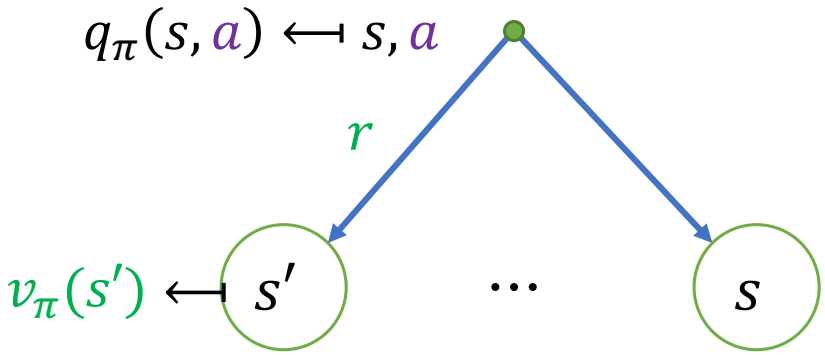
$$q_\pi(s, a) = \mathbb{E}_\pi\left(\mathcal{R}_{t+1} + \gamma q_\pi(s_{t+1}, a_{t+1}) | s_t = s, a_t = a\right)$$

Bellman Expectation Equation for v_π



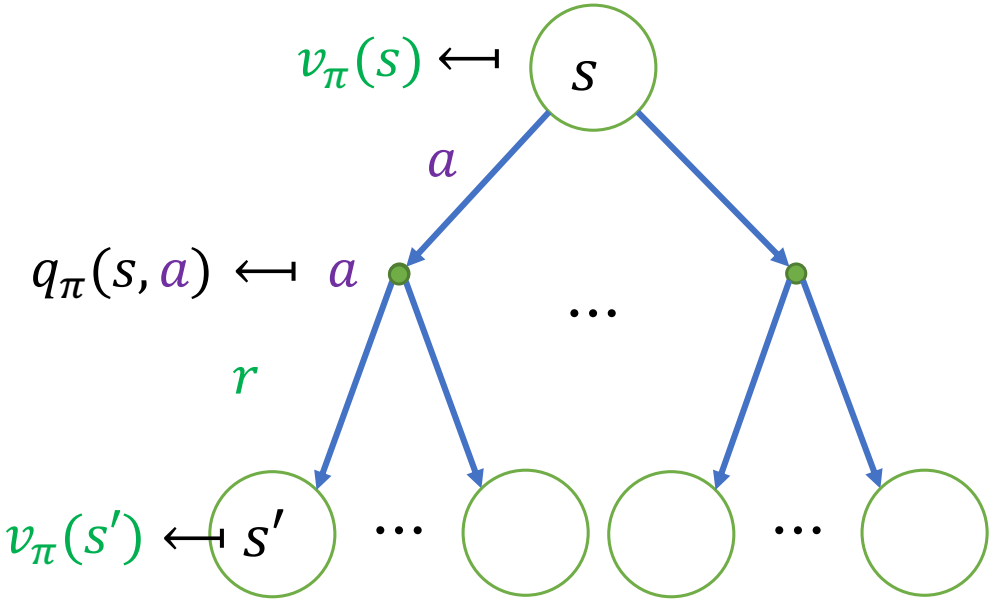
$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s)\, q_\pi(s,a)$$

Bellman Expectation Equation for q_π



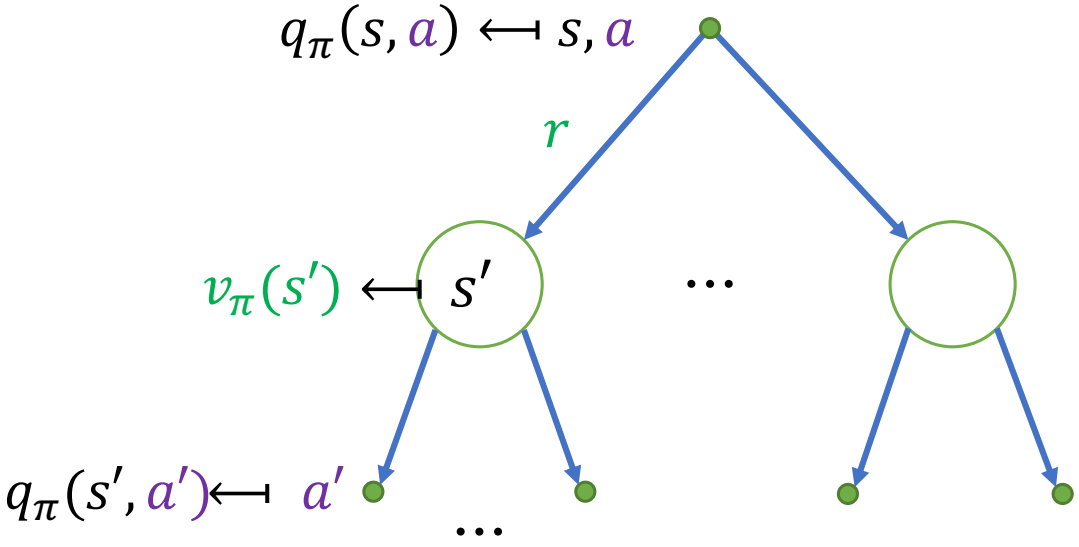
$$q_\pi(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{s,s'}^a \, v_\pi(s')$$

Recursive Formulation for v_π



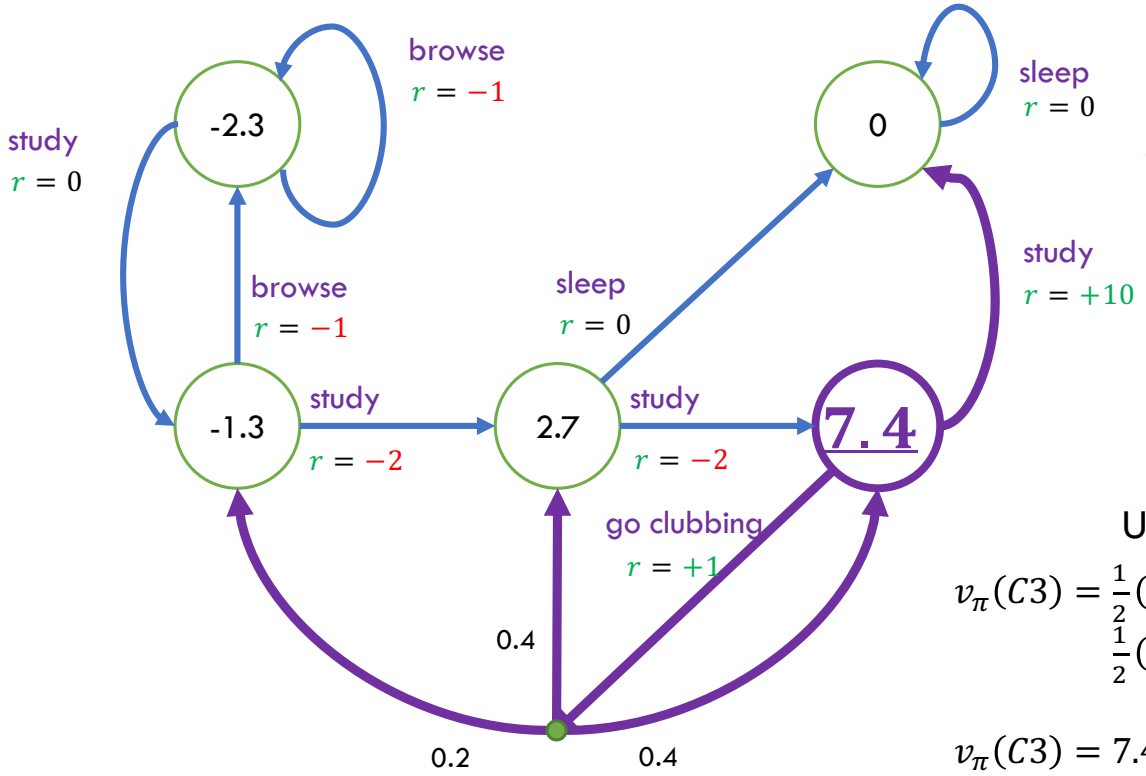
$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \underbrace{\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{s,s'}^a \, v_\pi(s')}_{q_\pi(s,a)}$$

Recursive Formulation for q_π



$$q_\pi(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{s,s'}^a \underbrace{\sum_{a' \in \mathcal{A}} \pi(a'|s')\, q_\pi(s', a')}_{v_\pi(s')}$$

Example Bellman Expectation for Student MDP



Under a uniform policy

$$v_\pi(C3) = \frac{1}{2}(10) + \frac{1}{2}(1 + 0.2\times -1.3 + 0.4\times 2.7 + 0.4\times 7.4)$$

$$v_\pi(C3) = 7.4$$

Bellman Expectation Equation (Matrix Form)

The Bellman expectation equation can be expressed concisely using the induced MRP:

$$v_\pi = \mathcal{R}^\pi + \gamma \mathcal{P}^\pi v_\pi$$

with the direct solution:

$$v_\pi = (I - \gamma \mathcal{P}^\pi)^{-1} \mathcal{R}^\pi$$

Optimal Value Functions

CS3244 Machine Learning

Optimal Value Function

The optimal state-value function $v_*(s)$ is the maximum state-value function over all policies:

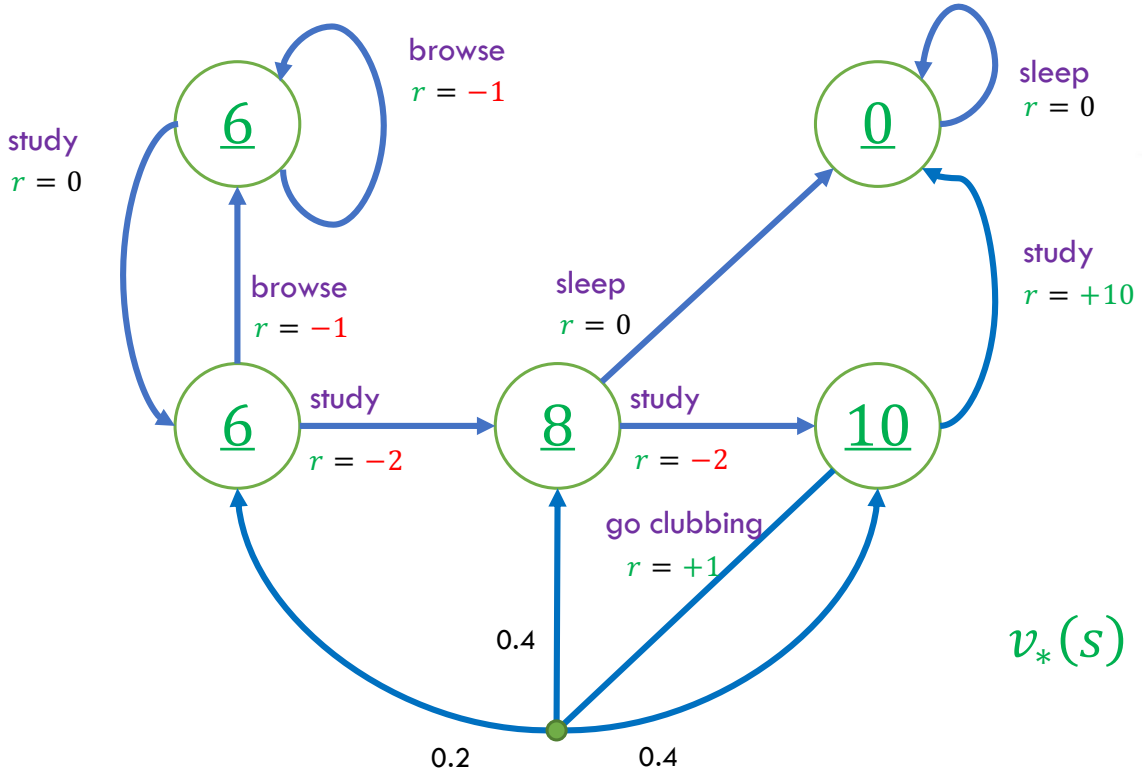
$$v_*(s) = \max_{\pi} v_\pi(s)$$

The optimal action-value function $q_*(s, a)$ is the maximum action-value function over all policies:

$$q_*(s, a) = \max_{\pi} q_\pi(s, a)$$

- These optimal functions specify the best possible performance in the MDP.
- We say that an MDP is solved when we know the optimal value functions.

Example: Optimum Value Function for Student MDP



$v_*(s)$ for $\gamma = 1$

Optimal Policy

Define a partial ordering over the possible policies:

$$\pi \geq \pi' \qquad \text{if} \qquad v_\pi(s) \geq v_{\pi'}(s), \forall s$$

<u>Theorem</u>: For any MDP,

- There exists an optimal policy π_* that is better than or equal to all other policies, $\pi_* \geq \pi, \forall \pi$
- All optimal policies achieve the optimal value function $v_{\pi_*}(s) = v_*(s)$
- They also achieve the optimal action-value function $q_{\pi_*}(s, a) = q_*(s, a)$

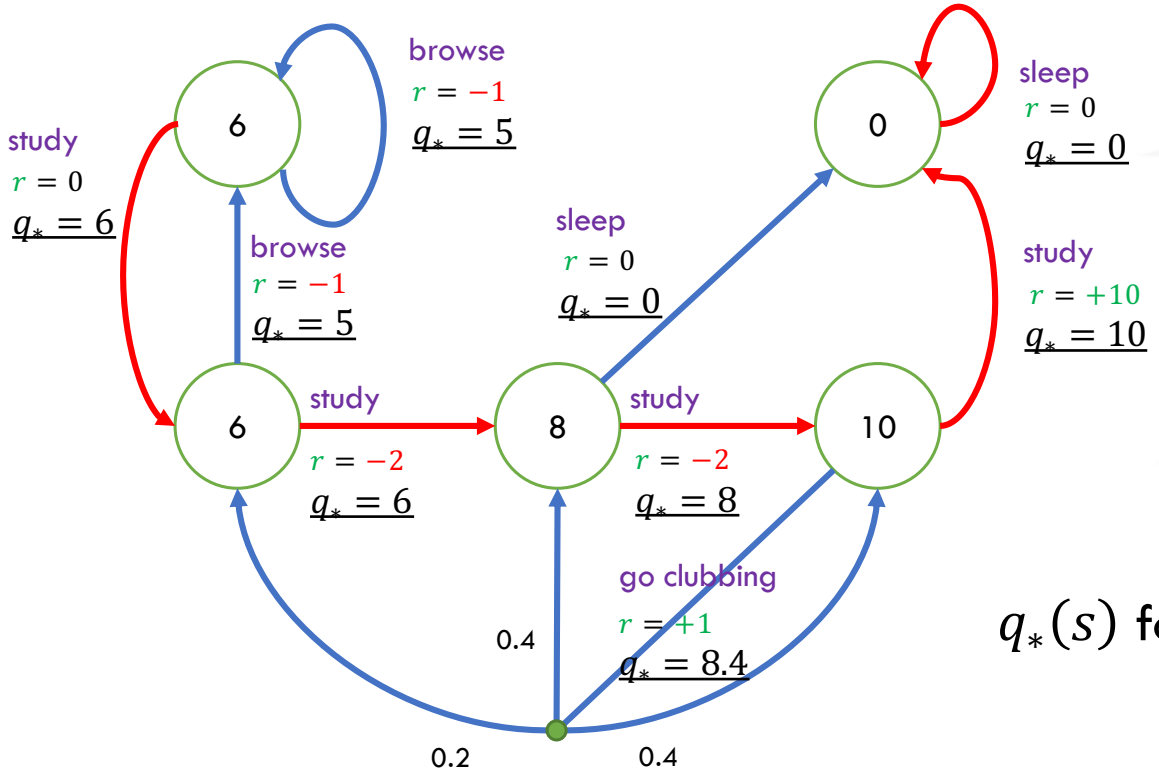
Finding an Optimal Policy

An optimal policy can be found by maximising over $q_*(s, a)$

$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = \underset{a \in \mathcal{A}}{\operatorname{argmax}}\, q_*(s, a) \\ 0 & \text{otherwise} \end{cases}$$

- There is always a deterministic optimal policy for any MDP
- If we know $q_*(s, a)$, we immediately have the optimal policy

Example Student Action-Value Function for Student MDP



⚠️⚠️ This diagram is not equivalent to the previous ones, as the rewards are now different on the arcs.

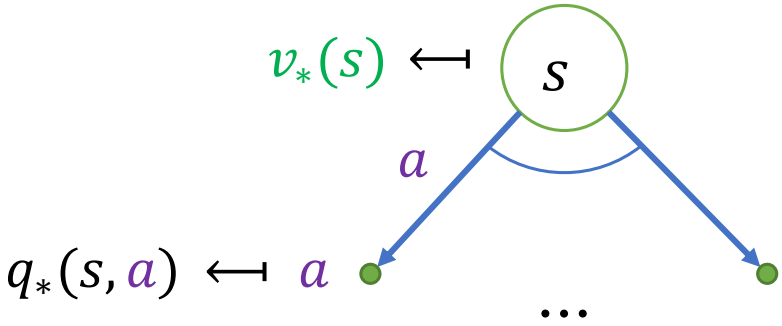
$q_*(s)$ for $\gamma = 1$

Bellman Optimality

CS3244 Machine Learning

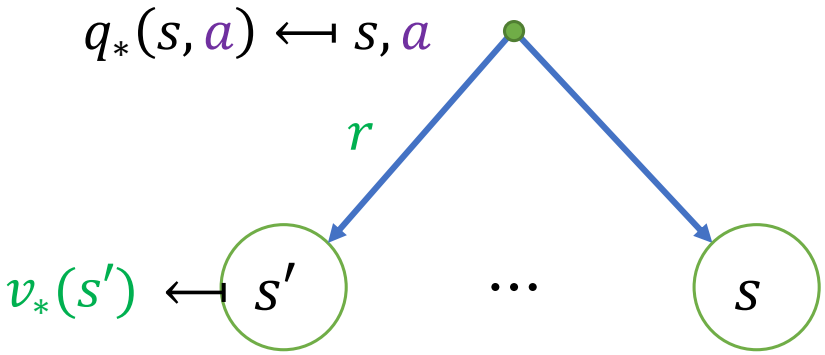
Bellman Optimality Equation for V^*

The optimal value functions are recursively related by the Bellman optimality equations:



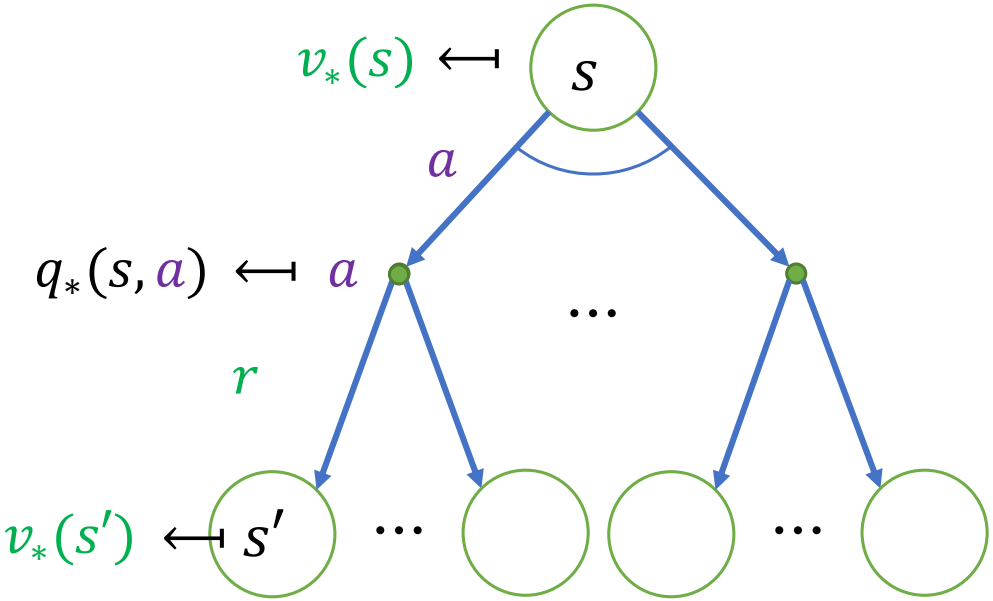
$$v_*(s) = \max_a q_*(s,a)$$

Bellman Optimality Equation for Q^*



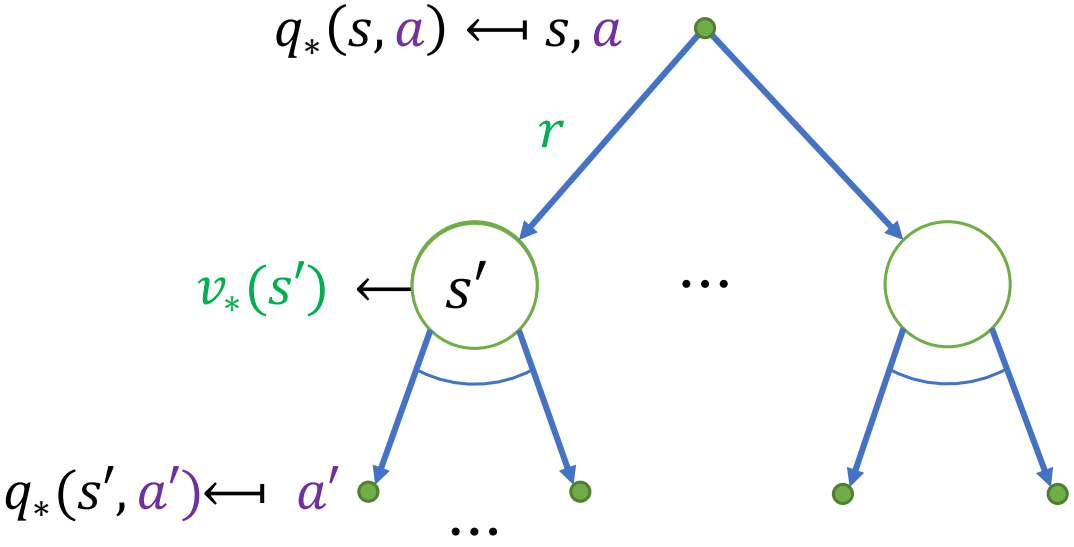
$$q_*(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{s,s'}^a \, v_*(s')$$

Recursive Formulation for V^*



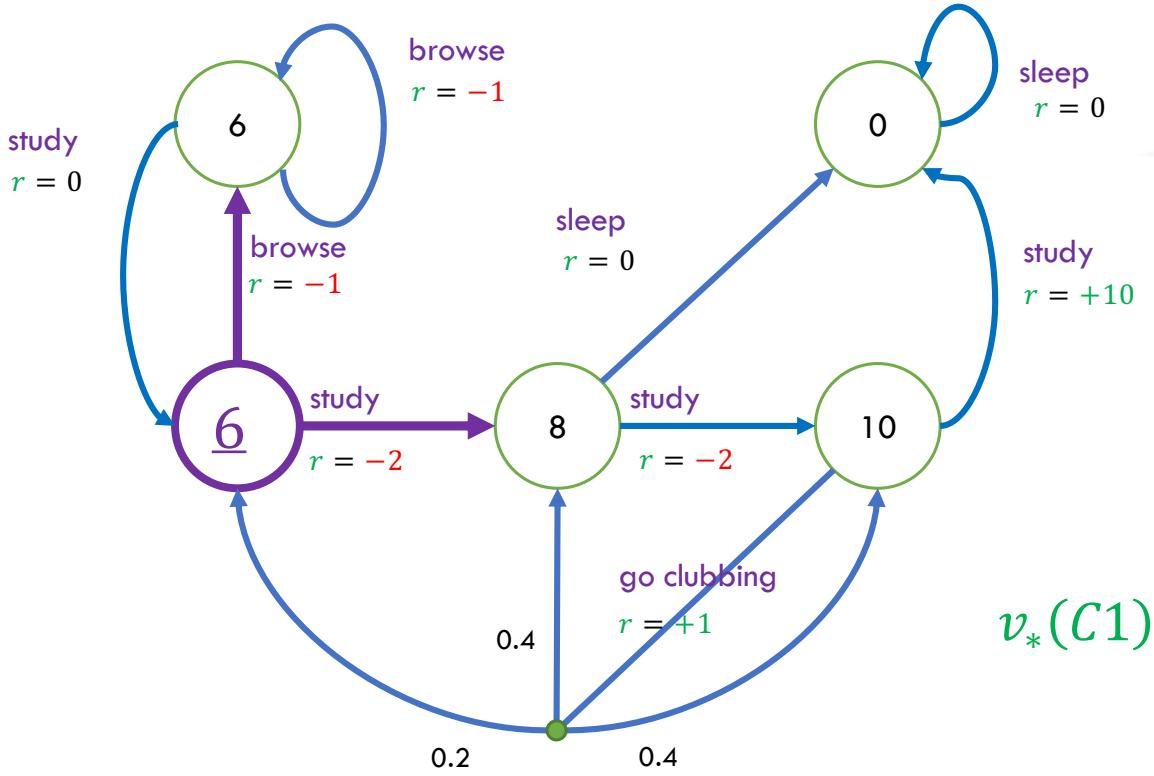
$$v_*(s) = \max_a \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{s,s'}^a \, v_*(s')$$

Recursive Formulation for Q^*



$$q_*(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{s,s'}^a \max_{a'} q_*(s', a')$$

Example Optimality Equation in Student MDP



$$v_*(C1) = \max(-2 + 8, -1 + 6)$$
$$= 6$$

Solving the Bellman Optimality Equation

- Bellman Optimality Equation is non-linear
- No closed form solution (in general)

Many iterative solution methods

- Value Iteration
- Policy Iteration
- Q-learning